

Divide and Translate: Parameter Isolation with Encoder Freezing for Low-Resource Indic NMT

IJCNLP–AAACL 2025 | [Link](#)

Author: Vaibhav Kanojia

Institution: Delhi Technological University (DTU)

Area: Efficient Architectures, Multilingual NLP, Representation Learning

Core Idea & Approach

Transformer-based multilingual models rely on **fully shared parameters** across languages and tasks. In extreme low-resource settings, this assumption breaks: shared models often exhibit **negative transfer**, where high-resource patterns dominate and degrade performance on structurally different languages.

To study this, I propose **Divide and Translate**, a simple alternative that enforces *parameter isolation*:

- Start from a pretrained multilingual model (NLLB-600M)
- **Freeze the encoder** to preserve cross-lingual semantic representations
- Train **separate, direction-specific decoders** for each translation task

This design treats the encoder as a **stable representation layer**, while decoders act as **specialized morphological adapters**, learning target-specific syntax (e.g., SOV vs SVO) without interference from other tasks.

To make this work under strict resource constraints:

- I use **bitext reversal augmentation** to double training data without introducing noise
- Apply **mixed-precision (bfloat16)** and **gradient checkpointing** to enable full training on a single 16GB GPU
- Carefully structure training to avoid leakage and stabilize optimization in <20k data regimes

This setup isolates where sharing helps (representation) and where it hurts (generation), allowing controlled analysis of architectural tradeoffs.

Results (MMLoSo 2025):

- **171.4 / 161.1** leaderboard score with <6% generalization gap
- Outperforms:
 - Unified fine-tuning → gradient interference across syntactic systems
 - LoRA/adapters → insufficient capacity for structural specialization

Future Directions (MSR Alignment)

I am interested in extending this perspective to **retrieval systems under production constraints**, where transformer encoders face a similar **accuracy–latency tradeoff**.

- **Retrieval-first encoder architectures**
Design alternatives to full self-attention that better match retrieval's needs: fast encoding with strong semantic similarity.
- **Decoupled representation and scoring**
Separate shared embedding layers from lightweight, task-adaptive scoring modules to reduce interference and inference cost.
- **Training objectives aligned with retrieval**
Move beyond language modeling toward objectives that directly optimize ranking quality, robustness to compression, and stability under distribution shift.
- **Bridging accuracy and efficiency**
Study why deep encoders outperform shallow ones, and whether distillation or better inductive biases can close this gap without inheriting transformer inefficiencies.

What would an encoder look like if it were designed for retrieval first, rather than inherited from language modeling?